

More Repetition

CSCI 1101

2024-03-01

While Loops

Last time we met the **while loop**.

```
while <guard> :  
    <loop_body>
```

where the **guard** is any boolean expression and the **loop body** is any nonempty sequence of statements (i.e. a **block**).

The loop body is executed repeatedly for as long as the guard remains `True`.

Picking and Prompting for Numbers

With the help of the `random` module, we can pick a random number between 1 and 10:

```
import random

def pick() :
    return random.randint(1 , 10)
```

We've seen how to prompt the user to enter a number:

```
def get_num() :
    answer = ""
    while not answer.isdigit() :
        answer = input("Please enter a number: ")
    return int(answer)
```

Activity: Number Guessing

Write a function `guessing_game` that picks a random number between 1 and 10 and has the user guess it.

For example:

```
> guessing_game()  
I'm thinking of a number between 1 and 10.  
Try to guess it!  
Please enter a number: 5  
too high, guess again!  
Please enter a number: 2  
too low, guess again!  
Please enter a number: 3  
You got it, the number was 3!
```

List iteration with `while`

A `while` loop is more general than a `for` loop, in that we can simulate

```
for item in list :  
    <do_something_with_item>
```

with

```
i = 0  
while i < len(list) :  
    item = list[i]  
    <do_something_with_item>  
    i += 1
```

We can choose a different pattern of iteration by changing the initial value and update function for the list index `i`.

Activity: Alternating Case

Use a `while` loop to write a function

`alternate_case : (str) --> str`

that takes a string and returns the result of alternating the case of each successive letter.

For example:

```
> alternate_case("bowdoin")  
'BoWdOiN'
```

```
> alternate_case("it was the best of times")  
'It wAs tHe bEsT Of tImEs'
```

while vs. for

In many cases you can choose whether to iterate over a list using either `while` or `for`.

A `while` loop gives you more control, but also more responsibility.

You must calculate and access the correct list element for each iteration of the loop, and make sure that the loop terminates.

A `for` loop iterates in a simple, predictable pattern.

The loop variable is assigned and the loop is terminated automatically. But you may need to “preprocess” the list in order to extract the information that you need.

Activity: Alternating Case (again)

Rewrite the `alternate_case` function using a `for` loop.

Hint: use the `enumerate` function.

Zippping Lists

In fact, the `enumerate` function is just a special case of another function:

```
zip : (list a , list b) --> iterator (tuple [a, b]).
```

This takes a pair of lists and “zips” them together into (morally) a list of pairs:

```
> zip(list('Bowdoin') , list('College'))
```

Using `zip` we can write our own `enumerate` function:

```
def my_enumerate(items) :  
    ''' sig: (list a) --> iterator (tuple [int, a]) '''  
    return zip(range(len(items)) , items)
```

To Do

Finish *Homework 6: More For-loops* on CodeRunner by 11:00AM next Thursday.

Finish on *Project 2 - What Does the Fox Say?* on CodeRunner by next Friday.